



Ng-Template IN ANGULAR



codewithrutu_09

✓ What is ng-template?

ng-template is a declarative Angular element used to define HTML blocks that aren't rendered by default.

✚ It's used for:

- Conditional rendering
- Reusable UI fragments
- Dynamic views (e.g. in `ngIf`, `ngFor`, `ngTemplateOutlet`)

📦 Think of it as a placeholder template that Angular can activate when needed.

✓ Basic Usage Example

```
app.component.html

<ng-template>
  <p>This will not render unless explicitly told.</p>
</ng-template>
```

! Nothing appears on the page unless Angular is instructed to render this using structural directives or programmatically.

✓ ng-template with *ngIf

You can customize what to show when a condition is false using ng-template.

```
app.component.html

<div *ngIf="isLoggedIn; else guestBlock">
  Welcome back, user!
</div>

<ng-template #guestBlock>
  <p>Please log in to continue.</p>
</ng-template>
```

💡 Use #templateRef to name a template and else to reference it.

✓ ng-template with ngFor

Render items using a custom template:

```
app.component.html

<ng-template let-item let-i="index">
  <li>{{ i + 1 }}. {{ item }}</li>
</ng-template>
```

Used with:

```
app.component.html

<ul>
  <ng-container *ngFor="let item of items; index as i;
  template: itemTemplate">
    </ng-container>
</ul>
```

Allows template customization for each iteration.

✓ ngTemplateOutlet – Dynamic Rendering

Use ngTemplateOutlet to render templates programmatically:

```
app.component.html

<ng-container *ngTemplateOutlet="dynamicTemplate"></ng-container>

<ng-template #dynamicTemplate>
  <p>This is a dynamic block!</p>
</ng-template>
```

🌟 Great for component design systems, dynamic UIs, and reusability.

✓ Passing Data to ng-template

Pass context (data) into the template:

```
app.component.html

<ng-container *ngTemplateOutlet="tpl; context: { $implicit:
'Angular' }"></ng-container>

<ng-template #tpl let-topic>
  <p>Welcome to {{ topic }} World!</p>
</ng-template>
```

🧠 \$implicit binds the value to the default let- variable.

Use Cases for ng-template

Powerful scenarios:

- **Display different UIs based on state (loading, error, success)**
- **Reuse template blocks (modals, table rows, custom cards)**
- **Create dynamic content systems**
- **Improve readability in complex conditions**

Key Takeaways

- **ng-template defines content that's not rendered by default**
- **Use with *ngIf, *ngFor, and ngTemplateOutlet**
- **Can be reused, passed with context, and conditionally shown**
- **Essential for building clean, maintainable, dynamic UIs**



codewithrutu_09



Like this post!



Share with your friends



Save it for later



**Tell us what you think in
comment**



codewithrutu_09



Rutuja Shinde